

Let's write!

A relay for Bluesky

*High-throughput networking
with Rust and Linux*

Alex Sayers

Bluesky

Like Twitter, but data publically available
⇒ you can look up someone's tweets

Bluesky

Like Twitter, but data publically available
⇒ you can look up someone's tweets

Also: “the firehose”

Bluesky

Like Twitter, but data publically available
⇒ you can look up someone's tweets

Also: “the firehose”

`ws://jetstream2.us-west.bsky.network/subscribe`

(“jetstream”: A variant of the firehose; omits signatures etc.)

Websockets

▸ Handshake

```
GET /subscribe HTTP/1.1
```

```
Connection: Upgrade
```

```
Upgrade: websocket
```

```
Sec-WebSocket-Version: 13
```

```
Sec-WebSocket-Key: MTIzNDU2Nzg5MDEyMzQ1Ngo=
```

Websockets

▸ Handshake (response)

```
HTTP/1.1 101 Switching Protocols
```

```
Connection: Upgrade
```

```
Upgrade: websocket
```

```
Server: jetstream
```

```
Sec-WebSocket-Accept: UzQo2NzMDEyM1NggMTIzND5=
```

```
<81>~^A<F6>{"did":"did:plc:5rg4sbiiiesfe563zzqumetx","time_us":"1763689821448290","kind":"commit","commit":  
{"rev":"3m64660t3d72m","operation":"create","collection":"app.bsky.feed.like","rkey":"3m64660stj72m","record":  
{"$type":"app.bsky.feed.like","createdAt":"2025-11-21T01:50:21.386Z","subject":{"cid":"bafyreiatvsmobksfekpb677zhwjkbxgpawpluzeduuglzyq2wpyjk64a","uri":"at://did:plc:o7ozygoaxci2  
djvrazhdmgs/app.bsky.feed.post/3m6466aia7s2u"}}, {"cid":"bafyreiguqrq7phuhl5tzyf2ek5pgaea7p4gtsrmyfmb7r3a4surjmf5qi"}}}  
  
<81>~^A<F6>{"did":"did:plc:htmhzec2p7aru5juw43732am","time_us":"1763689821448774","kind":"commit","commit":  
{"rev":"3m64660b3d72p","operation":"create","collection":"app.bsky.feed.like","rkey":"3m64660aomx2p","record":  
{"$type":"app.bsky.feed.like","createdAt":"2025-11-21T01:50:20.741Z","subject":{"cid":"bafyreidyg66ngsfvzp5xgzw6vde2okbhker54k4bndx7ilz5rrdv545rk4","uri":"at://did:plc:upogmzimjos5wbp4y  
5dkzt7x/app.bsky.feed.post/3m5zzb6nms22o"}}, {"cid":"bafyreiftxgsfr7revfoafmrbugks3ec5pkh2zfv6luyzdohys76yhtdxnq"}}}  
  
<81>~^C7{"did":"did:plc:nutrikdii7qo6xbb52mdlyf2","time_us":"1763689821449193","kind":"commit","commit":  
{"rev":"3m6466nzec12z","operation":"create","collection":"app.bsky.feed.post","rkey":"3m64666fhis2e","record":  
{"$type":"app.bsky.feed.post","createdAt":"2025-11-21T01:50:04.166Z","embed":{"$type":"app.bsky.embed.images","images":[{"alt":"","aspectRatio":{"height":1500,"width":2000},"image":  
{"$type":"blob","ref":{"$link":"bafkreifnxbxn4v1psdawnxktr7aoeyzocooux3b2uwxyp7avs7npubiam"},"mimeType":"image/jpeg","size":886994}],{"alt":"","aspectRatio":  
{"height":1500,"width":2000},"image":{"$type":"blob","ref":{"$link":"bafkreid4toh755jum2mh64ds3ehon77fom3y3qaamhr2ywnfhg4dibwkiu"},"mimeType":"image/jpeg","size":970465}}}], "langs":  
["ja"],"text":"乗った！"},"cid":"bafyreieccaisnrex4akqpyxzunnsfh16dl6y3142drko4ab4zhj6otxu"}}}  
  
<81>~^A<8A>{"did":"did:plc:hyhxd432dqwxzrurdymo2b5","time_us":"1763689821450390","kind":"commit","commit":  
{"rev":"3m6466ny6f72r","operation":"create","collection":"app.bsky.graph.follow","rkey":"3m6466nxnr2r","record":  
{"$type":"app.bsky.graph.follow","createdAt":"2025-11-21T01:50:20.390Z","subject":{"did:plc:iwqdab3ifz6ooighwshlpiiz"},"cid":"bafyreia3zmpvp2gig7brfcdqg7lkgoi6t4jbbxmd6ynr2rosndedpbkjme"}}}  
  
<81>~^A<FA>{"did":"did:plc:bgickrkqnnv7nyfh6y1bkxwb","time_us":"1763689821450842","kind":"commit","commit":  
{"rev":"3m6466ooca626","operation":"create","collection":"app.bsky.feed.repost","rkey":"3m6466ontlg26","record":  
{"$type":"app.bsky.feed.repost","createdAt":"2025-11-21T01:50:21.111Z","subject":{"cid":"bafyreiezm2hskvynr3ru775zy6hitmrjj27ow43ic4sx6vy53m22zjhki","uri":"at://did:plc:t5g4e7slz7mc56  
x1bcpobdvf/app.bsky.feed.post/3m63av4qtæ2m"}}, {"cid":"bafyreih3ekwujchuavh3xbqes12w67rrxjr2h2hyohz4pmsg26qtfnpbe"}}}  
  
<81>~^A<F6>{"did":"did:plc:cp4mjijntnhh27pdtqi7hucg","time_us":"1763689821451503","kind":"commit","commit":  
{"rev":"3m6466oom4a2j","operation":"create","collection":"app.bsky.feed.like","rkey":"3m6466oo4ia2j","record":  
{"$type":"app.bsky.feed.like","createdAt":"2025-11-21T01:50:20.931Z","subject":{"cid":"bafyreidynbpfmymrtfwcj6uax7pmmq7y35mv4ifc6kpaifv7fayvr2mty","uri":"at://did:plc:pbtde2womj2b6sbmt  
2usuzyw/app.bsky.feed.post/3m63rrmcryk2e"}}, {"cid":"bafyreiehlcciox934nc3mmcgxp5qndn6ullvlofmruwh75aiebwkgecw4"}}}  
  
<81>~^B<93>{"did":"did:plc:5segm472sboj3bnepzusiot","time_us":"1763689821452124","kind":"commit","commit":  
{"rev":"3m6466ogwke2b","operation":"create","collection":"app.bsky.feed.like","rkey":"3m6466ogi2b","record":  
{"$type":"app.bsky.feed.like","createdAt":"2025-11-21T01:50:20.918Z","subject":{"cid":"bafyreia32lc31f6mdsdyuxx6e555bjto2c5baegj5owg4cmmvcliicudq","uri":"at://did:plc:lrw4cnptopspeg5  
tthztcliw/app.bsky.feed.post/3m63g7tcnva2e"},"via":{"cid":"bafyreie6ofr46v6h2zuwud5gv3zlf2pvg5s7a3jivn6ik3sa5kjghvx4d"},"uri":"at://did:plc:rrfwrhud4ovela3oe6isre5/app.bsky.feed.  
repost/3m644ru253n25"}}, {"cid":"bafyreiggz52lgkwhyuo4wnf5qrdyldwqvmufunzcuvwrsp4glchcgz4fa"}}}
```

<81>~^A<F6>

```
{
  "did": "did:plc:5rg4sbiiiesfe563zzqumetx",
  "time_us": 1763689821448290,
  "kind": "commit",
  "commit": {
    "rev": "3m6466ot3d72m",
    "operation": "create",
    "collection": "app.bsky.feed.like",
    "rkey": "3m6466ostj72m",
    "record": {
      "$type": "app.bsky.feed.like",
      "createdAt": "2025-11-21T01:50:21.386Z",
      "subject": {
        "cid": "bafyreiatvsmobksfekpb677zhwjkbxgpawwpluzeduuglzyq2wpyjk64a",
        "uri": "at://did:plc:o7ozygoaxci2djvrazhdmgs/app.bsky.feed.post/3m6466aia7s2u"
      }
    },
    "cid": "bafyreiguqrq7phuh15tzyf2ek5pgeea7p4gtsrmymfb7r3a4surjmf5qi"
  }
}
```


Handling disconnects

▸ Handshake

```
GET /subscribe?cursor=1763689821448290 HTTP/1.1
```

```
...
```

Handling disconnects

▸ Handshake

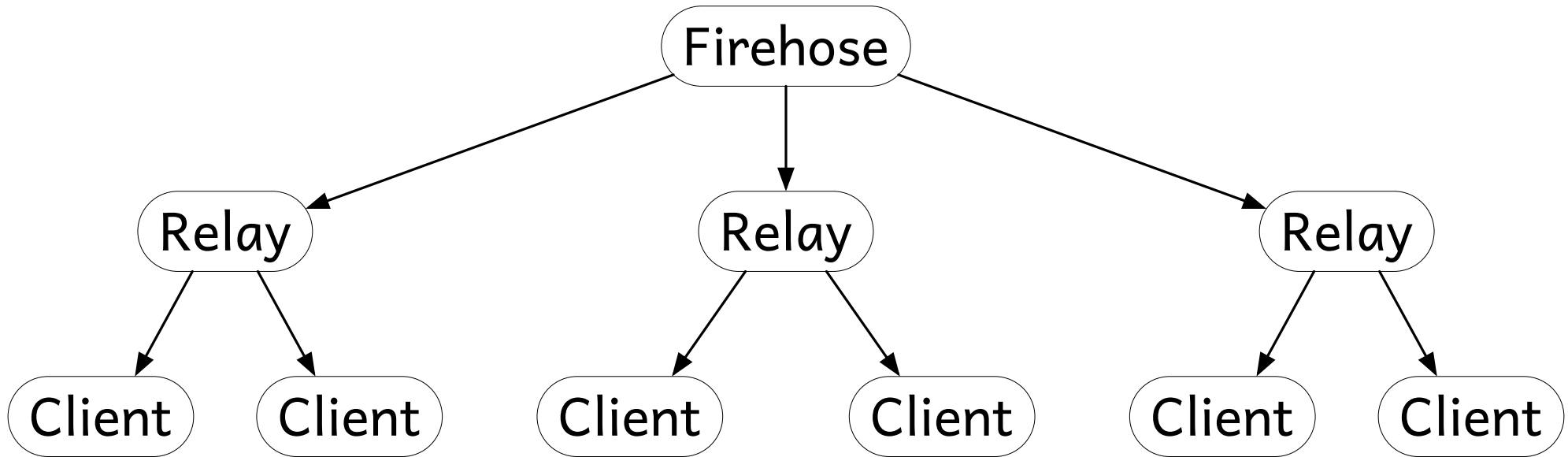
```
GET /subscribe?cursor=1763689821448290 HTTP/1.1  
...
```

Remember the last timestamp you saw
⇒ you can resume from where you were

Essential functionality

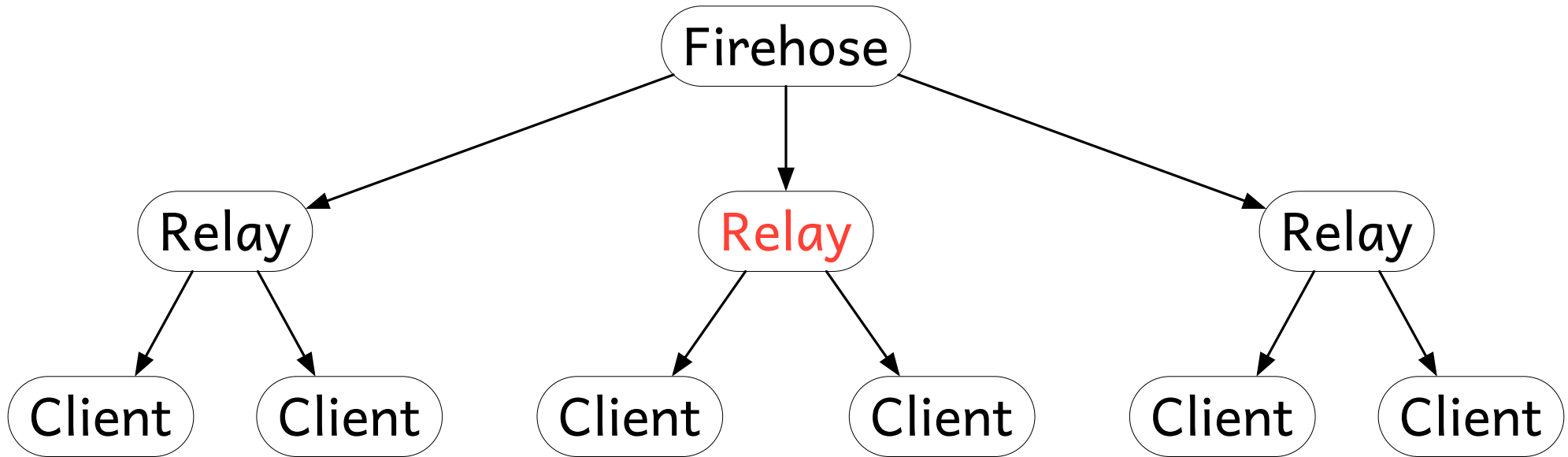
Relays

Data is distributed to clients via a network of relays



Relays

Data is distributed to clients via a network of relays



How much data?

~400 events/second

Typical event: ~0.5 KiB

Bitrate: ~200 KiB/s

How much data?

~400 events/second

Typical event: ~0.5 KiB

Bitrate: ~200 KiB/s

10Gbps NIC => ~6000 simultaneous clients

200Gbps NIC => ~120k simultaneous clients!

How much data?

~400 events/second

Typical event: ~0.5 KiB

Bitrate: ~200 KiB/s

10Gbps NIC => ~6000 simultaneous clients

200Gbps NIC => ~120k simultaneous clients!

...but official implementation can only do 1 Gbps 😞

```
static CLIENTS: Mutex<Vec<TcpStream>>;
```

▷ new client thread

```
loop {  
    let conn = listener.accept()?;  
    websocket_handshake(&conn);  
    CLIENTS.lock()?.push(conn);  
}
```



```
static CLIENTS: Mutex<Vec<TcpStream>>;
```

...

▷ new client thread

```
loop {  
    let conn = listener.accept()?;  
    websocket_handshake(&conn);  
    CLIENTS.lock()?.push(conn);  
}
```

▷ events thread

```
loop {  
    let event = recv_from_upstream();  
    for conn in CLIENTS.lock()? {  
        let frame = add_ws_framing(event);  
        conn.write_all(&frame)?;  
    }  
}
```

```
}  
}
```

```
static CLIENTS: Mutex<Vec<Receiver<Json>>>;
```

▷ new client thread

```
loop {  
    let conn = listener.accept()?;  
    thread::spawn(|| client_thread(conn));  
}
```

▷ events thread

```
loop {  
    let event = recv_from_upstream();  
    for tx in CLIENTS.lock()? {  
        tx.send(event);  
    }  
}
```

▷ per-client thread

```
websocket_handshake(&conn);  
let (tx, rx) = mpsc::channel();  
CLIENTS.lock()?.push(tx);  
while let Ok(event) = rx.recv() {  
    let frame = add_ws_framing(event);  
    conn.write_all(&frame)?;  
}
```

<81>~^A<F6>{"did":"did:plc:5rg4sbiiiesfe563zzqumetx","time_us":1763689821448290,"kind":"commit","commit":{"rev":"3m6466ot3d72m","operation":"create","collection":"app.bsky.feed.like","rkey":"3m6466ostj72m","record":{"\$type":"app.bsky.feed.like","createdAt":"2025-11-21T01:50:21.386Z","subject":{"cid":"bafyreiatvsmobksfekpb677zhwjkbxgpawpluzeduuglzyq2wpyjk64a","uri":"at://did:plc:o7ozygoaxci2djvrazhdmgs/app.bsky.feed.post/3m6466aia7s2u"}},"cid":"bafyreigyurqr7phuhl5tzyf2ek5pgeea7p4gtsrmyfb7r3a4surjmf5qi"}}<81>~^A<F6>{"did":"did:plc:htmhzec2p7aru5juw43732am","time_us":1763689821448774,"kind":"commit","commit":{"rev":"3m6466ob3d72p","operation":"create","collection":"app.bsky.feed.like","rkey":"3m6466oaomx2p","record":{"\$type":"app.bsky.feed.like","createdAt":"2025-11-21T01:50:20.741Z","subject":{"cid":"bafyreidyg66ngsfvzp5xgzv6vde2okbhker54k4bndx7ilz5rrdv545rk4","uri":"at://did:plc:upogmzimjos5wbp4y5dkzt7x/app.bsky.feed.post/3m5zzb6nms22o"}},"cid":"bafyreiftxgsfr7revfoafmrbuqks3ec5pkh2zfv16uyzdohys76yhtdxnq"}}<81>~^C7{"did":"did:plc:nutrikdii7qo6xbb52mdlyf2","time_us":1763689821449193,"kind":"commit","commit":{"rev":"3m6466nzec12z","operation":"create","collection":"app.bsky.feed.post","rkey":"3m64666fhis2e","record":{"\$type":"app.bsky.feed.post","createdAt":"2025-11-21T01:50:04.166Z","embed":{"\$type":"app.bsky.embed.images","images":[{"alt":"","aspectRatio":{"height":1500,"width":2000},"image":{"\$type":"blob","ref":{"\$link":"bafkreifnxbxn4vlpdawnxktr7aoeyzocooux3b2uwxyp37avs7npuibam"},"mimeType":"image/jpeg","size":886994}},{alt":"","aspectRatio":{"height":1500,"width":2000},"image":{"\$type":"blob","ref":{"\$link":"bafkreid4toh755jum2mh64ds3ehon77fom3y3qaamhr2ywnfhg4dibwkiu"},"mimeType":"image/jpeg","size":970465}}]},"langs":["ja"],"text":"乗った！"},"cid":"bafyreieccaisnrex4akqpyxzunn2sfhl6dl6y3l42drko4ab4zhj6otxu"}}<81>~^A<8A>{"did":"did:plc:hyhxd432dqwxzrurdymoj2b5","time_us":1763689821450390,"kind":"commit","commit":{"rev":"3m6466ny6f72r","operation":"create","collection":"app.bsky.graph.follow","rkey":"3m6466nxnr2r","record":{"\$type":"app.bsky.graph.follow","createdAt":"2025-11-21T01:50:20.390Z","subject":{"did:plc:iwqdab3ifz6ooighwhslpiiz"},"cid":"bafyreia3zmpvp2gig7brfcdqg7lkgoi6t4jbbxmd6ynr2rosndedpbkjme"}}<81>~^A<FA>{"did":"did:plc:bgickkrqnnv7nyfh6ylbkxwb","time_us":1763689821450842,"kind":"commit","commit":{"rev":"3m6466ooa626","operation":"create","collection":"app.bsky.feed.repost","rkey":"3m6466ontlg26","record":{"\$type":"app.bsky.feed.repost","createdAt":"2025-11-21T01:50:21.111Z","subject":{"cid":"bafyreiezm2hskvynr3ru775zy6hitmrjj27ow43ic4sx6vy53m22zjhki","uri":"at://did:plc:t5g4e7slz7mc56xlbcpobdvf/app.bsky.feed.post/3m63av4qtat2m"}},"cid":"bafyreih3e3wkujchuavh3xbqesl2w67rrxjrj2h2hyohz4pmg26qtfnpbe"}}<81>~^A<F6>{"did":"did:plc:cp4mjijntnhh27pdtqi7hucg","time_us":1763689821451503,"kind":"commit","commit":{"rev":"3m6466oom4a2j","operation":"create","collection":"app.bsky.feed.like","rkey":"3m6466oo4ia2j","record":{"\$type":"app.bsky.feed.like","createdAt":"2025-11-21T01:50:20.931Z","subject":{"cid":"bafyreidynbpfmymrtfwcj6uax7pmmq7y35mv4ifc6kpaifvh7fayvr2mty","uri":"at://did:plc:pbtde2womj

Multicast

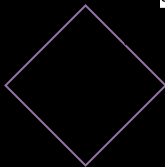
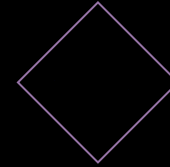
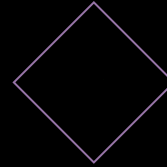
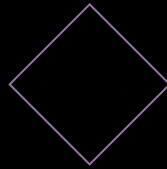
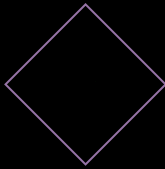
UDP multicast

Multicast over open internet: not a thing

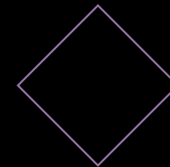
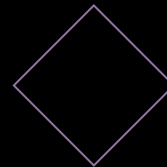
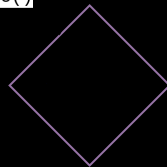
TCP multicast: not a thing



new event



sendfile()



Multicast


```
static BYTES: Mutex<Vec<u8>>;  
static CVAR: Condvar;
```

▷ events thread

```
loop {  
    let event = recv_from_upstream();  
    let frame = add_ws_framing(event);  
    let bytes = BYTES.lock()?;  
  
    bytes.extend(&frame)?;  
    CVAR.notify_all();  
}
```

```
static BYTES: Mutex<Vec<u8>>;  
static CVAR: Condvar;
```

▷ events thread

```
loop {  
    let event = recv_from_upstream();  
    let frame = add_ws_framing(event);  
    let bytes = BYTES.lock()?;  
  
    bytes.extend(&frame)?;  
    CVAR.notify_all();  
}
```

▷ new client thread

```
loop {  
    let conn = listener.accept()?;  
    thread::spawn(|| client_thread(conn));  
}
```

▷ per-client thread

```
websocket_handshake(&conn);  
let bytes = BYTES.lock()?;  
let mut cursor = bytes.len();  
loop {  
    CVAR.wait(bytes);  
    let n =  
conn.write_all(&bytes[cursor..])?;  
    cursor += n;  
}
```

```
static BYTES: Mutex<Vec<u8>>;
static CVAR: Condvar;
static INDEX: Mutex<BTreeMap<Timestamp,
usize>>;
```

▷ events thread

```
loop {
    let event = recv_from_upstream();
    let frame = add_ws_framing(event);
    let bytes = BYTES.lock()?;
    INDEX.lock().insert(event.ts,
bytes.len());
    bytes.extend(&frame)?;
    CVAR.notify_all();
}
```

▷ new client thread

```
loop {
    let conn = listener.accept()?;
    thread::spawn(|| client_thread(conn));
}
```

▷ per-client thread

```
let timestamp = websocket_handshake(&conn);
let bytes = BYTES.lock()?;
let mut cursor =
INDEX.lock()?.get(timestamp);
loop {
    CVAR.wait(bytes);
    let n =
conn.write_all(&bytes[cursor..])?;
```

```
cursor += n;
```

```
}
```

write()ing to a TCP socket

- Copy bytes from userspace buffer to socket's `sk_write_queue`
- May return once the bytes are safely appended to queue
 - ▶ May take the chance to do some socket work
 - Which might include actually sending those bytes
- Bytes in the queue will be sent, possibly multiple times
- When an ACK arrives, bytes are discarded from the front of the queue
- How is data stored in the queue?

Fragments

An arbitrary-length arbitrary-offset area of memory which resides within a 0 or higher order page. Multiple fragments within that page are individually refcounted, in the page's reference counter.

— mm/page_frag_cache.c

```
struct page_frag {  
    struct page *page;  
    __u16 offset;
```

```
    __u16 size;  
};
```

If you've ever used the “bytes” crate this may look familiar

There's also an optimization which allows very small fragments to be inlined into the skb

Page allocator [OLD]



Page allocator [OLD]



Page allocator [OLD]



Page allocator [OLD]



Page allocator [OLD]



You can split a block into two smaller blocks

You can merge a block with its “buddy” to form a bigger block

=> an allocation’s size is always a power-of-two

Min size: 4 KiB (*typically*)

Max size: 4 MiB (*typically*)

Folios [OLD]

Allocation + refcount *+ flags + lru + mapping/index + ...*

...actually two refcounts:

- kernelspace
- userspace (*called the “mapcount”*)

Refcount goes to zero => add to freelist and merge up

If refcount > 1 => COW (shared)

Fragments [OLD]

```
struct page_frag {  
    struct folio *folio;  
    __u16 offset;  
    __u16 size;  
};
```

Used everywhere, eg.

```
type Pipe = (VecDeque<Fragment>, WaitQueue);
```

Writing to a TCP socket [OLD]

Every TCP connection has a “send queue”

Write to socket $\Rightarrow$ push data onto the back

Ack arrives $\Rightarrow$ discard data from the front

Writing to a TCP socket [OLD]

Every TCP connection has a “send queue”

Write to socket $\Rightarrow$ push data onto the back

Ack arrives $\Rightarrow$ discard data from the front

```
type SendQueue = LinkedList<Fragment>
```


Writing to a TCP socket [OLD]

Every TCP connection has a “send queue”

Write to socket $\Rightarrow$ push data onto the back

Ack arrives $\Rightarrow$ discard data from the front

```
type SendQueue = LinkedList<Fragment>
```

When you call `write()`, the kernel needs to get your data into a (kernel-owned) page

A file's page cache [OLD]

Represented by a data-structure called an `xarray`.

Divides the file into 4 KiB chunks

Maps the chunks to pages of memory (refcounted)

```
HashMap<ChunkIdx, Arc<[u8; 4096]>>
```

Source of truth for a file's contents. Whenever you interact with a file, you're interacting with its page cache

sendfile [OLD]

```
sendfile(sock, file, offset, len)
```

Go to the file's page cache, look up the desired range

=> list of page slices

Go to the socket's send queue, push slices onto the back

That's it.

```
static BYTES: Mutex<Vec<u8>>;
static CVAR: Condvar;
static INDEX: Mutex<BTreeMap<Timestamp,
usize>>;
```

▷ events thread

```
loop {
    let event = recv_from_upstream();
    let frame = add_ws_framing(event);
    let bytes = BYTES.lock()?;
    INDEX.lock().insert(event.ts,
bytes.len());
    bytes.extend(&frame)?;
    CVAR.notify_all();
}
```

▷ new client thread

```
loop {
    let conn = listener.accept()?;
    thread::spawn(|| client_thread(conn));
}
```

▷ per-client thread

```
let timestamp = websocket_handshake(&conn);
let bytes = BYTES.lock()?;
let mut cursor =
INDEX.lock()?.get(timestamp);
loop {
    CVAR.wait(bytes);
    let n =
conn.write_all(&bytes[cursor..])?;
```

```
cursor += n;
```

```
}
```

```
static FILE: Mutex<File>;  
static FILE_LEN: AtomicUsize;
```

▷ new client thread

```
loop {  
    let conn = listener.accept()?;  
    thread::spawn(|| client_thread(conn));  
}
```



```
static FILE: Mutex<File>;  
static FILE_LEN: AtomicUsize;
```

```
FILE_LEN.fetch_add(n);  
}
```

▷ new client thread

```
loop {  
    let conn = listener.accept()?;  
    thread::spawn(|| client_thread(conn));  
}
```

▷ events thread

```
loop {  
    let event = recv_from_upstream();  
  
    let frame = add_ws_framing(event);  
    let n = FILE.write_all(&frame)?;
```



```
static FILE: Mutex<File>;  
static FILE_LEN: AtomicUsize;
```

```
FILE_LEN.fetch_add(n);  
}
```

▷ new client thread

```
loop {  
    let conn = listener.accept()?;  
    thread::spawn(|| client_thread(conn));  
}
```

▷ events thread

```
loop {  
    let event = recv_from_upstream();  
  
    let frame = add_ws_framing(event);  
    let n = FILE.write_all(&frame)?;
```

▷ per-client thread

```
websocket_handshake(&conn);  
let mut buf = vec![0; 4096];  
loop {  
    if cursor < FILE_LEN.load() {  
        FILE.read_at(&mut buf, cursor)?;  
        let n = conn.write_all(&buf)?;  
        cursor += n;  
    } else {  
        thread::sleep(10);  
    }  
}
```

```
static FILE: Mutex<File>;  
static FILE_LEN: AtomicUsize;
```

```
FILE_LEN.fetch_add(n);  
}
```

▷ new client thread

```
loop {  
    let conn = listener.accept()?;  
    thread::spawn(|| client_thread(conn));  
}
```

▷ events thread

```
loop {  
    let event = recv_from_upstream();  
  
    let frame = add_ws_framing(event);  
    let n = FILE.write_all(&frame)?;
```

▷ per-client thread

```
websocket_handshake(&conn);

loop {
    if cursor < FILE_LEN.load() {

        let n = sendfile(sock, file, cursor,
usize::MAX)?;
        cursor += n;
    } else {
        thread::sleep(10);
    }
}
```

```
static FILE: Mutex<File>;  
static FILE_LEN: AtomicUsize;  
static INDEX: Mutex<HashMap<Timestamp,  
Offset>>;
```

```
let n = FILE.write_all(&frame)?;  
FILE_LEN.fetch_add(n);  
}
```

▷ new client thread

```
loop {  
    let conn = listener.accept()?;  
    thread::spawn(|| client_thread(conn));  
}
```

▷ events thread

```
loop {  
    let event = recv_from_upstream();  
    INDEX.insert(FILE_LEN.load(), event.ts);  
    let frame = add_ws_framing(event);
```

▷ per-client thread

```
let timestamp = websocket_handshake(&conn);
let cursor = INDEX.lookup(timestamp);
loop {
    if cursor < FILE_LEN.load() {

        let n = sendfile(sock, file, cursor,
usize::MAX)?;
        cursor += n;
    } else {
        thread::sleep(10);
    }
}
```

Thundering herd

`CVAR.notify_all()`

=> thousands of threads fight over the mutex

io_uring

Hole punch